

Agent Skills Can Be Harmful: An Empirical Study of Skill-Induced Failures in LLM Agents

Gen Dong¹, Yanjie Gao^{2,†}, Liquan Li³, Tianyin Xu⁴, Yu Hua¹, Fan Yang²

¹Huazhong University of Science and Technology

²Microsoft Research

³Microsoft

⁴University of Illinois Urbana-Champaign

Abstract—Agent skills are the *de facto* mechanism for extending LLM agents with reusable guidance. A skill can shape the agent’s task execution, including planning, tool use, problem-solving, and validation. Prior work reported mixed results of agent skills: some skills improve task success rates, while others have no effect, increase token use and execution time, and even reduce success rates. This paper presents a comprehensive analysis of skill-induced agent failures by attributing task failures and cost regressions to specific loaded skills. We introduce a differential analysis framework that attributes a failure or regression to a skill by comparing a target skill-guided run against a no-skill or semantically matched-skill reference run that solves the same task, or solves it more cheaply. We instantiate this framework on SkillsBench and SWE-Skills-Bench, yielding 307 skill-induced failures, including 125 functional failures and 182 efficiency regressions. We also build SKILLTRIAGE, a taxonomy-guided attribution tool that normalizes paired cases, extracts differential evidence, and produces triage reports. Our major findings include: (1) Skill-induced functional failures are rarely caused by obviously irrelevant skills; instead, seemingly relevant skills often make the agent incorrectly implement or omit task-required implementation elements. Task-Implementation Fault accounts for 86 of 125 functional failures (68.8%), while wrong artifact locations and environment mismatches account for 24 and 13 cases, respectively. (2) Skill-induced efficiency regressions are not explained by prompt length alone. When regressions come from context overhead, they are almost entirely caused by mandatory skill-body text (43 of 46 context-overhead cases), but overall Excessive Procedure dominates with 114 of 182 cases (62.6%). (3) The largest sources within Excessive Procedure are excessive verification and heavy implementation pipelines, contributing 67 and 30 cases, respectively. This shows that skills often turn validation checklists and construction recipes into mandatory work. Based on our findings, we propose research topics and tooling improvements for safer and more cost-aware skill reuse.

I. INTRODUCTION

LLM agents are increasingly used to solve complex software-engineering and other knowledge-intensive tasks by combining model reasoning with tools, repositories, and external environments. To make procedural guidance reusable across tasks, recent systems introduce *skills*: instruction packages that encode task-specific procedures, examples, constraints, and tool-use guidance. Once selected, a skill is

inserted into the agent context and can shape planning, file edits, command execution, validation, and stopping decisions.

However, this mechanism also creates a risk. A skill may be topically relevant but operationally wrong for the concrete task: it may imply an incompatible path, over-prescribe a workflow, induce an incompatible dependency or runtime state, or encourage unnecessary verification. Such guidance can induce a functional failure when the task fails, or an efficiency regression when the task passes only with substantially higher token use or longer execution time. These failures are especially problematic because skills are intended to be reused, so a harmful skill can repeatedly bias future agent executions.

Recent skill benchmarks suggest that this risk is not hypothetical. SkillsBench reports average gains from curated skills but also observes pass-rate drops on 16 of 84 tasks [1]. SWE-Skills-Bench reports limited average improvement, many skills with no pass-rate gain, and cases with token overhead up to 451% [2]. These results show that skills can help, harm, or waste resources depending on the task and skill content.

However, existing evaluations leave four gaps that make skill-induced failures difficult to study. First, the central challenge is not merely detecting failed runs, but isolating and attributing failures to the loaded skill. A failed with-skill run alone cannot distinguish skill-induced harm from base-agent limitations, verifier scope, ordinary agent variance, or environmental artifacts. Identifying skill-induced failures therefore requires contrastive evidence. Second, existing skill benchmarks are designed to measure utility rather than uncover failure mechanisms. Consequently, their original skill settings provide limited negative cases for studying skill-induced failures. Third, aggregate pass rates and cost ratios provide outcome-level metrics but do not explain how a skill changes an agent trajectory. Related studies on long context, irrelevant context, and prompt sensitivity explain broad context effects [3]–[6]. However, they do not identify how a specific skill document causes a verifier-facing functional failure or a hidden efficiency regression. Finally, even when failure attribution is performed manually, it does not scale to agent platforms or skill marketplaces where new skills and skill updates must be screened continuously.

In this paper, we present a comprehensive empirical study of skill-induced agent failures in LLM agents. Inspired by differential testing [7], we construct paired target/reference

This work was done during Gen Dong’s internship at Microsoft Research.

[†]Corresponding author.

executions that use a no-skill or semantically matched skill execution as a pseudo-oracle. The design attributes a failure to a skill only when the same task is otherwise solvable or can be solved more cheaply under this paired-comparison setup. We instantiate the study on SkillsBench and SWE-Skills-Bench, augment the original benchmark settings with semantically matched public skills, and analyze task specifications, skill contents, execution trajectories, verifier outcomes, token use, and execution time. We then derive root-cause taxonomies for functional failures and efficiency regressions, and build SKILLTRIAGE, a taxonomy-guided tool that normalizes paired cases, extracts differential evidence, and produces attribution reports for triage.

Specifically, our study addresses the following research questions:

1) **RQ1: How can we construct a contrastive dataset of skill-induced agent failures for root-cause analysis?**

To answer this question, we design a differential-testing-motivated contrastive construction that pairs each target skill-guided execution with a no-skill or semantically matched-skill reference run as a pseudo-oracle. Using our contrastive labeling criteria, we confirm 307 skill-induced failures, including 125 functional failures and 182 high-confidence efficiency regressions. A detailed description is provided in Section III.

2) **RQ2: What are the root causes of skill-induced functional failures?**

To answer this question, we manually analyze task specifications, skill contents, execution trajectories, verifier outputs, and artifacts for the confirmed functional failures. Our study reveals several notable findings. For example, most functional failures are not caused by obviously irrelevant skills. Instead, seemingly relevant skills often make the agent implement a required field, API behavior, calculation, output format, or domain rule incorrectly or omit it entirely. Task-Implementation Fault accounts for 86 of 125 functional failures (68.8%), while verifier-facing artifact and environment boundaries account for 24 and 13 cases, respectively. A detailed analysis is provided in Section IV.

3) **RQ3: What are the root causes of skill-induced efficiency regressions?**

To answer this question, we analyze target/reference pairs whose token use and execution time both increase under the loaded skill, with at least one metric exceeding the primary $T = 2.0$ threshold. Our study reveals that high-confidence efficiency regressions are dominated by Excessive Procedure rather than prompt length alone. At the primary $T = 2.0$ threshold, Excessive Procedure accounts for 114 of 182 cases (62.6%), driven mainly by excessive verification (67 cases) and heavy implementation pipelines (30 cases). A detailed analysis is provided in Section V.

4) **RQ4: How can we automatically attribute skill-induced agent failures to their root causes?**

To answer this question, we build SKILLTRIAGE, a taxonomy-

guided attribution tool that normalizes paired cases, extracts differential evidence, and produces triage reports. For functional failures and efficiency regressions, it matches manually assigned exact root causes for 111/125 (88.8%) and 132/182 (72.5%), respectively. A detailed evaluation is provided in Section VI.

In summary, this paper makes the following contributions:

- 1) We identify a critical and timely problem in LLM-agent skill reuse: skills can cause functional failures or substantial efficiency regressions even when they are topically relevant. We construct a contrastive analysis dataset of 307 confirmed skill-induced failures from SkillsBench and SWE-Skills-Bench, where each case is validated by no-skill or semantically matched-skill reference runs and annotated as a functional failure or efficiency regression.
- 2) We present the first comprehensive study of skill-induced failures in LLM agents, categorizing functional failures and efficiency regressions and analyzing their root causes.
- 3) We develop a tool, SKILLTRIAGE, which normalizes paired cases, extracts differential evidence, and attributes skill-induced agent failures to root causes. We demonstrate the practical utility and effectiveness of SKILLTRIAGE through automated attribution evaluation.

II. BACKGROUND

a) Agent skills in LLM agents: An *agent skill* is a structured package of procedural knowledge that augments an LLM agent at inference time without modifying the model parameters [8]. In current agent systems, a skill is usually organized around a named `SKILL.md` file. The skill framework selects one or more skills for a task and loads their contents into the agent context. Once loaded, a skill can influence the agent’s planning, repository exploration, tool use, code edits, tests, and stopping decision.

b) Skill contents: A skill typically contains four kinds of information. First, metadata such as the skill name, description, frontmatter, or when-to-use text signals when the skill applies. Second, the main body gives procedural guidance, including task-solving steps, implementation rules, workflow constraints, and checklists. Third, supplementary files may provide examples, templates, schemas, scripts, reference documents, or other reusable resources loaded on demand. Fourth, validation guidance may suggest tests, expected outputs, debugging steps, or completion criteria. These components matter because they become part of the agent’s effective task context rather than passive documentation.

Figure 1 illustrates the kind of information that a skill can inject. The description provides an applicability signal; repository and module wording can shape where the agent writes artifacts; dependency instructions can alter the execution environment; and verification checklists can add procedure cost. Thus, a skill can help by providing reusable knowledge, but it can also mislead the agent when its assumptions do not match the concrete task.

```

<!-- Metadata: -->
---
name: rag-backend-helper
description: Use for backend RAG applications.
---
<!-- Skill name: -->
# RAG Backend Helper
<!-- Procedure body: -->
1. Inspect the repository structure and identify the app
   package.
2. Install or import vector-store and document-loader
   packages.
3. Add retrieval code under the app's retriever module.
<!-- Validation guidance: -->
4. Run the full test suite and fix all failures before
   stopping.
<!-- Supplementary material: -->
See examples/rag_template.py for a reference implementation.

```

Fig. 1. A simplified SKILL.md example showing typical skill components.

c) *Agent Execution Trajectories*: An execution trajectory is the ordered sequence of model interactions, tool invocations, and intermediate actions produced by an agent while attempting a task. A trajectory may include model responses, tool calls, file operations, searches, code edits, command executions, and other observable interactions with the environment. Throughout this paper, root-cause analysis is performed based on execution trajectories rather than solely on final task outcomes.

III. METHODOLOGY

A. Study Design

We use a differential-testing-inspired contrastive design to attribute failures to loaded skills rather than to baseline model limitations, task difficulty, or ordinary differences between repeated executions. Figure 2 illustrates the setup: paired executions keep the task, verifier, agent framework, model, repository or container state, and input data fixed, and vary only the skill setup, including no skill and candidate skills. Each run produces two observable outcomes: a correctness outcome, measured by the task verifier or tests, and a cost outcome, measured by token use and execution time.

We define the run being audited as the *target run*. A *reference run* is another execution of the same task under a different skill setup, either no skill or a semantically matched skill. Throughout the paper, PASS and FAIL denote the deterministic verifier outcome of a single run. For a paired comparison, FAIL/PASS means that the target run fails and the reference run passes, while PASS/PASS means that both runs pass. We use two audit settings throughout the paper. A *with/no-skill comparison* compares a target run executed with the audited skill against a reference run on the same task without any skill. A *cross-skill comparison* compares the target run against another semantically matched skill setup on the same task. For functional failures, the target run fails while the reference run passes; for efficiency regressions, both runs pass but the target run is substantially more expensive. The reference run acts as a pseudo-oracle, not as a ground-truth solution: it shows that the same task can be solved, or solved more cheaply, under the same task and verifier conditions. With task, verifier, repository, model, and agent framework fixed, only the skill setup varies; therefore, when the reference

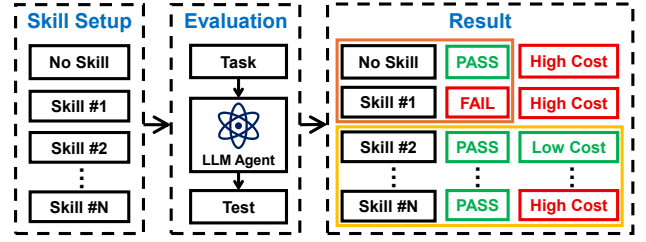


Fig. 2. Differential evaluation across skill setups.

run passes or solves the task more efficiently, it provides contrastive evidence for attributing the difference to the loaded skill under controlled conditions.

We define two failure classes:

- **Functional failure**: the target run fails the verifier while a reference run passes.
- **Efficiency regression**: both the target and reference runs pass, but the target run has substantially more token use, longer execution time, or both.

In Figure 2, the orange box highlights the functional failure pattern: the no-skill run passes, but the audited skill run fails on the same task. The yellow box highlights the efficiency regression pattern: two runs both pass, but the audited skill run incurs substantially higher cost than the reference run.

Figure 3 summarizes our four-stage methodology. First, we select skill benchmarks with deterministic verifiers and executable task environments, so that pass/fail and cost outcomes can be measured reproducibly rather than decided by manual judgment after the run. Second, we augment these benchmarks with semantically matched public skills, broadening the skill-comparison space and making it possible to observe a wider range of skill-induced failures. Third, we execute the augmented benchmark tasks with the agent under controlled skill setups and collect runtime evidence, including trajectories, verifier results, token use, and execution time. Finally, we label functional failures and efficiency regressions from the paired execution data and refine the dataset by removing ambiguous, verifier-narrow, or duplicate cases before root-cause analysis.

B. Study Subjects

Our study uses two representative skill benchmarks as study subjects. SkillsBench provides 84 evaluated tasks across 11 domains with no-skill, curated-skill, and self-generated-skill settings [1]. SWE-Skills-Bench provides 490 repository-based software-engineering task instances with no-skill and curated-skill settings [2]. Both benchmarks use deterministic programmatic verifiers, enabling stable pass/fail and cost comparisons.

The original benchmark settings are valuable starting points because they provide executable tasks, deterministic verifiers, and curated skills, but they are not sufficient by themselves for mechanism-level failure attribution. Their primary goal is to measure skill utility rather than to mine failure mechanisms, so the comparison space is narrow: SkillsBench compares no-skill, curated-skill, and self-generated-skill conditions, while SWE-Skills-Bench compares no-skill and curated-skill conditions. These settings can show that a skill helps, hurts, or adds

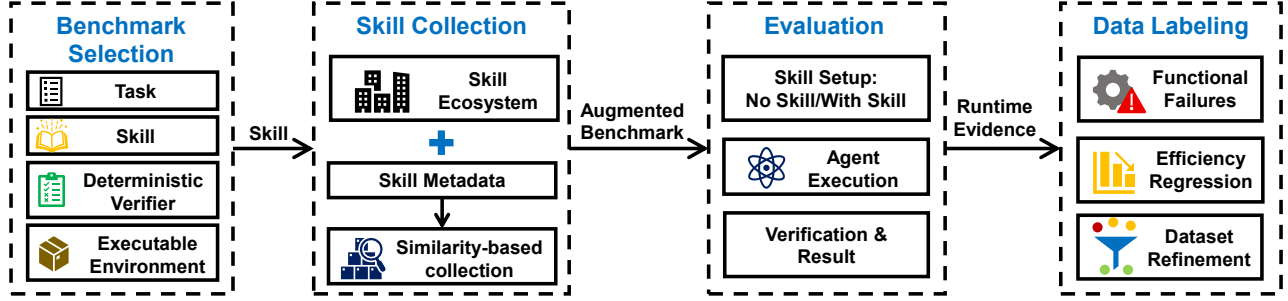


Fig. 3. Overview of the study methodology.

cost, but they often do not provide an alternative successful skill for the same task, which is needed to localize the failure to differences in skill content and induced trajectories.

C. Data Collection

To broaden the skill setups available for paired evaluation beyond the original benchmark settings, we search two public skill-sharing sites that host reusable agent skills: `smithery.ai` and `skillsmp.com`. We use these sites because their skills include agent-visible names and descriptions, which allow us to retrieve plausible alternatives to the curated benchmark skills. For each curated benchmark skill, we query the sites with the curated skill’s name and description, embed the curated and candidate skill metadata with `all-MiniLM-L6-v2`, a widely used sentence-embedding model for semantic-similarity matching, and retain up to the top-5 candidates with cosine similarity of at least 0.7 to filter out off-topic matches while limiting evaluation cost. We refer to these retained metadata-similar candidates as semantically matched public skills.

The retrieved skills are used as candidate skill setups, not as mandatory context loaded regardless of the task. During execution, the agent receives the task and the available candidate skill package under the normal skill-loading interface; it may choose to load the skill when it judges the skill relevant, or avoid relying on it when the task does not match. This design reduces the risk that an unrelated retrieved skill is artificially forced into the agent context, while still expanding the space of plausible skill alternatives for contrastive analysis. For SkillsBench, the original self-generated-skill condition is included in the original comparison space but not treated as a variant in the expanded public-skill comparison space, because our expansion focuses on reusable public skills that users are more likely to select and load rather than benchmark-specific self-generated skills.

Together with the original curated skill, these retained public variants give up to six skill variants per task. For a task with k available skill variants, with/no-skill comparisons contribute k no-skill/with-skill pairs, and cross-skill comparisons contribute $k(k-1)$ ordered skill/skill pairs because each skill variant can define the audited target run while another variant defines the reference run. In the original benchmark settings, SkillsBench has two with-skill conditions per task, while SWE-Skills-Bench has one; after expansion, both benchmarks have up to

TABLE I
POTENTIAL PAIRED COMPARISONS BEFORE EVALUATION.

Benchmark	Pair type	Original	Expanded
SkillsBench	With/no-skill	168	504
SkillsBench	Cross-skill	168	2,520
SWE-Skills-Bench	With/no-skill	490	2,940
SWE-Skills-Bench	Cross-skill	0	14,700
Total		826	20,664

six skill variants per task. Table I summarizes the resulting comparison space before evaluation. The augmentation therefore increases the comparison space from 826 to 20,664 potential paired comparisons, a roughly $25\times$ expansion, making mechanism-level failure attribution feasible.

We execute tasks with OpenCode 1.15.1 [9] and Claude Opus 4.6 [10] under with/no-skill and cross-skill comparison settings. This configuration combines a widely used open-source agent runtime with a state-of-the-art frontier language model, providing a representative setup for contemporary LLM agents. Each pair fixes the task instruction, repository or container state, data files, verifier, agent framework, and model; only the skill setup changes. For each run, we record the loaded skill, trajectory, verifier outcome, token use, and execution time.

D. Data Labeling

a) *Functional failures*: We include two types of functional failures in the final analysis dataset. A with/no-skill-comparison functional failure requires the no-skill run to pass and the audited-skill run to fail. A cross-skill functional failure requires the target run with the audited skill to fail and a run with another semantically matched skill to pass.

b) *Efficiency regressions*: For efficiency regressions, we only compare PASS/PASS pairs, where both the target and reference runs pass. Let r_{tok} be the target-to-reference token ratio and r_{time} be the target-to-reference execution-time ratio. We classify a PASS/PASS pair as an efficiency regression at threshold T iff:

$$\min(r_{\text{tok}}, r_{\text{time}}) > 1.0 \wedge \max(r_{\text{tok}}, r_{\text{time}}) > T.$$

That is, both token use and execution time must regress relative to the reference run, and at least one must more than double ($T = 2.0$) relative to the reference run. Requiring both ratios to be greater than 1.0 excludes token–time tradeoffs, where one metric increases while the other decreases. We use $T = 2.0$ as the primary threshold to focus on large

TABLE II
FAILURE DATASET CONSTRUCTION FROM LABELED CANDIDATES TO
FINAL ANALYSIS CASES.

Failure Type	Audit setting	Labeled candidates	Analysis cases
Functional failure	With/no-skill	70	38
	Cross-skill	245	87
	Subtotal	315	125
Efficiency regression	With/no-skill	159	128
	Cross-skill	191	54
	Subtotal	350	182
Total		665	307

cost regressions and reduce sensitivity to small token/time fluctuations. We label each efficiency regression as *token-dominant*, *time-dominant*, or *joint token-and-time* according to which ratio exceeds T .

c) *Dataset refinement*: After data collection, we identify functional failures and efficiency regressions from the executed pairs in the expanded comparison space as labeled candidates. From the 20,664 potential paired comparisons summarized in Table I, the executed evaluations yield 665 labeled candidates: 315 functional failure candidates and 350 efficiency regression candidates. We remove candidates with insufficient evidence, likely verifier-induced false positives, and duplicate same-task/same-skill effects. When the same failure appears in both with/no-skill and cross-skill audits, we keep the with/no-skill instance. After refinement, we manually inspect each remaining case, assign one root-cause label, and finalize the labels through group consensus. Table II summarizes the count flow from labeled candidates to final analysis cases. For functional failures, the with/no-skill audit starts from 70 automatically identified with-skill-FAIL/no-skill-PASS candidate records: 50 from SWE-Skills-Bench and 20 from SkillsBench. The cross-skill audit starts from 245 failed with-skill records in tasks where at least one skill run passes and another skill run fails: 131 from SWE-Skills-Bench and 114 from SkillsBench. For efficiency regressions, labeled candidates are threshold-triggering cost-regression candidates identified under the primary threshold $T = 2.0$ before dataset refinement: 159 from with/no-skill audits and 191 from cross-skill audits.

Together, the final analysis dataset contains 307 confirmed skill-induced failures: 125 functional failures and 182 high-confidence efficiency regressions.

IV. WHAT ARE THE ROOT CAUSES OF FUNCTIONAL FAILURES?

This section presents the categorization of the 125 confirmed functional failures in the final analysis dataset. Table III summarizes the functional-failure taxonomy, which consists of four high-level categories: Applicability Mismatch (APM), Environment Mismatch (EM), Task-Implementation Fault (TIF), and Artifact Misplacement (AM). For categories with multiple subcategories, the table also reports their subcategory breakdowns. Each functional failure receives exactly one subcategory label, and category counts aggregate the subcategories that belong to the same mechanism family. For categories with a single subcategory, Table III reports the

category-level row directly. Among the four categories, Task-Implementation Fault is the largest, accounting for 86 cases (68.8%), followed by Artifact Misplacement with 24 cases (19.2%).

A. Applicability Mismatch (APM)

This category accounts for 2 confirmed functional failures (1.6%) where the skill should not have been applied under the concrete task scope. These failures originate from the skill-loading stage: the skill’s name, description, frontmatter, or when-to-use framing makes the agent treat the skill as applicable even though its guidance does not match the concrete task scope. This category covers cases where skill metadata gives an inaccurate applicability signal. For example, a skill written for building application-level RAG backends can make the agent treat a LangChain library-maintenance task as outside the skill’s intended scope, causing it to avoid creating the task-required example files. The decisive problem is not the final artifact path, but the decision to apply the skill under the wrong task scope.

B. Environment Mismatch (EM)

This category accounts for 13 confirmed functional failures (10.4%) where the skill prescribes or induces an execution environment that differs from the verifier’s environment. These failures are not primarily about what artifact should be built, but about where and under which runtime state the artifact is executed.

Broken Dependency or Runtime (BDR) appears in 5 cases (4.0%), where the skill explicitly recommends or depends on a package, runtime, import path, binary, or external component that fails in the task environment.

Environment-State Mismatch (ESM) represents 8 cases (6.4%) where the skill indirectly leads the agent to change package resolution, working directory, installed versions, local installs, or tool state, so the target run validates under an environment state that the evaluation environment does not share.

A representative ESM case appears in an `openpyxl` task:

```
Task:
create 'openpyxl/utils/report_engine.py'

Verifier output:
test_report_engine_importable
test_engine_generates_valid_xlsx_file
python -c "from openpyxl.utils.report_engine import *; print('OK!'"

Target trajectory:
"Old openpyxl version incompatible ... I'll install a modern version"
pip install openpyxl
cd /tmp
sys.path = [p for p in sys.path if '/workspace/openpyxl' not in p]

Reference-run trajectory:
patches 'openpyxl/formatting/rules.py' in the repository
'from collections import Mapping' -> 'from collections.abc import Mapping'
```

The verifier imports from the repository package, while the target run validates against an external install. This target/reference comparison shows that the failure is caused by an environment-state mismatch rather than by the artifact content alone.

TABLE III
CLASSIFICATION OF THE 125 CONFIRMED FUNCTIONAL FAILURES.

Category / Subcategory		Definition	Count	Percentage
Applicability Mismatch		Skill metadata gives an incomplete or misleading applicability signal, causing the agent to apply the skill to a task where its guidance is inappropriate.	2	1.6%
Environment Mismatch	Broken Dependency or Runtime	The skill explicitly recommends or depends on a dependency or runtime component that fails in the task environment, causing installation, import, or execution failure.	5	4.0%
	Environment-State Mismatch	The skill indirectly leads the agent to change package resolution, working directory, version, installation state, or tool state, so the agent's self-check environment state differs from the evaluator's state.	8	6.4%
	<i>Subtotal</i>		13	10.4%
Task-Implementation Fault	Obstructive Workflow Guidance	The skill induces excessive exploration, setup, or procedural steps that prevent the agent from producing the required artifact within the available budget.	4	3.2%
	Incorrect Required-Element Fill	The skill leads the agent to implement a task-required element with an incorrect method, API, value, policy, or output structure.	46	36.8%
	Required-Element Omission	The skill leads the agent to leave a task-required element or dependent step absent, leaving the artifact incomplete without providing a concrete substitute.	36	28.8%
	<i>Subtotal</i>		86	68.8%
Artifact Misplacement		The skill leads the agent to write or integrate the required artifact at a location different from the task-specified path or integration point.	24	19.2%
Total			125	100.0%

C. Task-Implementation Fault (TIF)

This is the largest category, accounting for 86 confirmed functional failures (68.8%). These failures occur when an on-topic skill induces a fault in the implementation process or generated artifact, preventing the agent from satisfying a concrete task-required implementation element. In many cases, the agent over-trusts a topically matched skill and treats its reusable defaults, examples, or templates as task-specific requirements. The skill may make the workflow too heavy to finish, implement a task-required element incorrectly, or leave a required element absent.

Obstructive Workflow Guidance (OWG) occurs in 4 cases (3.2%), where otherwise relevant guidance induces excessive exploration, setup, audits, or procedural checks that prevent the agent from producing the required artifact within the budget.

Incorrect Required-Element Fill (IRF) is the largest subcategory in this category, with 46 cases (36.8%), and means that the target run implements a task-required element with an incorrect method, API, value, policy, or output structure.

Required-Element Omission (RRO) includes 36 cases (28.8%) where the target run leaves a required field, option, behavior, domain rule, validation, or dependent step absent. The distinction between Incorrect Required-Element Fill (IRF) and Required-Element Omission (RRO) is whether the target artifact contains a concrete but wrong implementation of the required element, or leaves that element absent.

A representative IRF case appears in a spreadsheet task:

Task:
calculate net exports as percent of GDP

Verifier output:
values are about 100x too small

Target trajectory:
(Exports - Imports) / GDP

Reference-run trajectory:
(Exports - Imports) / GDP * 100

The target fills the required calculation with the wrong representation. The reference run matters because it distinguishes a skill-induced failure from ordinary arithmetic failure: the target run uses a bare ratio, whereas the reference run keeps the task-required percentage scaling.

A representative RRO case appears in a RAG demo task:

Task:
chunk size, overlap, top-k, and model parameters must be configurable

Verifier output:
test_model_parameters_configurable

Target trajectory:
CLI args: --question, --chunk-size, --chunk-overlap, --top-k

Reference-run trajectory:
RAGConfig includes model_name = "gpt-3.5-turbo"

The target leaves a required implementation element absent. Here the skill example shows retrieval parameters such as chunk size, overlap, and top-k, but does not show how to configure model parameters. The target run follows that partial example: it exposes the retrieval parameters through CLI flags, but omits the required model-parameter field that the reference run adds as model_name.

Finding 1: Only 2 of 125 functional failures are classified as Applicability Mismatch (1.6%), while most fall under Task-Implementation Fault, indicating that on-topic skills more often induce task-implementation faults in required implementation elements. Task-Implementation Fault accounts for 86 of 125 functional failures (68.8%), and Incorrect Required-Element Fill (IRF) plus Required-Element Omission (RRO) account for 82 cases.

Implication: Skill authors should separate mandatory task requirements from examples, defaults, reusable templates, and optional workflows, so that agents do not mistake reusable guidance for task-specific obligations.

D. Artifact Misplacement (AM)

This category accounts for 24 confirmed functional failures (19.2%) where the agent builds a plausible artifact but writes or integrates it at the wrong location. The target run often follows repository or package-layout conventions instead of the task-specified output path. This category covers cases where the skill leads the agent to write or integrate the required artifact at a location different from the task-specified path or integration point.

A representative case appears in a LangChain RAG task:

```
Task:
create files under 'libs/langchain/langchain/'
- retrievers/hybrid_retriever.py
- text_splitter/semantic_chunker.py
- chains/rag_chain.py

Verifier output:
file-path existence tests fail for 'libs/langchain/langchain/...'

Target trajectory:
"The package is 'langchain_classic'."
"I'll use 'langchain_classic' since that's the real package"
Writes under 'libs/langchain/langchain_classic/'

Reference-run trajectory:
"I'll create the files at the paths specified in the task"
Writes under 'libs/langchain/langchain/'
```

The decisive failure is artifact placement, not the plausibility of the generated RAG components.

Finding 2: A substantial share of functional failures occur at execution-surface boundaries, where the skill changes the verifier-observed environment state or artifact location. Environment Mismatch accounts for 13 cases (10.4%), while Artifact Misplacement accounts for 24 cases (19.2%).
Implication: Agent platforms should treat task-specified paths, integration points, package state, and working directories as guarded constraints before accepting skill-guided environment changes or repository-native path substitutions.

V. WHAT ARE THE ROOT CAUSES OF EFFICIENCY REGRESSIONS?

This section presents the categorization of the 182 high-confidence efficiency regressions in the final analysis dataset under the primary $T = 2.0$ threshold. We use $T = 2.0$ as a conservative high-confidence threshold: a run must more than double token use or execution time while the other metric also increases relative to the reference run, which separates substantial skill-induced overhead from token-time tradeoffs and ordinary run-to-run variation. Table IV summarizes the efficiency-regression taxonomy, which consists of three high-level categories: Context Bloat (CO), Excessive Procedure (EP), and Dependency Resolution (DO). For categories with multiple subcategories, the table also reports their subcategory breakdowns. Each efficiency regression receives exactly one subcategory label, and category counts aggregate subcategories in the same cost-mechanism family. For categories with a single subcategory, Table IV reports the category-level row directly. Among the three categories, Excessive Procedure is the largest, accounting for 114 cases (62.6%), followed by Context Bloat with 46 cases (25.3%). The three categories

correspond to three cost layers: larger per-call context, additional trajectory steps, and dependency-repair work.

A. Context Bloat (CO)

This category accounts for 46 high-confidence efficiency regressions (25.3%) where the skill increases the cost of model calls rather than substantially changing the task-solving trajectory. The agent may follow a similar sequence of actions as the reference run, but pays more tokens because the skill body or skill-referenced material remains in context.

Skill-Body Context Bloat (SBCB) is the dominant subcategory in this category, with 43 cases (23.6%), and refers to cases where the skill body itself adds enough text to make each model call substantially more expensive.

Supplementary-Material Bloat (SMB) appears in 3 cases (1.6%), where the skill directs the agent to load or inspect auxiliary references, templates, examples, or documentation, expanding the context or tool-output volume. This imbalance reflects how skills are loaded during execution: the main skill body is repeatedly present in the agent context, while supplementary material is only costly when the agent explicitly reads or follows it. As a result, always-loaded skill text creates a broader cost surface than optional reference material. This category suggests that skill authors should shorten mandatory skill bodies and lazy-load optional references only when the task actually requires them.

Finding 3: When efficiency regressions arise from context overhead, the overhead is almost entirely caused by mandatory skill-body text. Skill-Body Context Bloat (SBCB) accounts for 43 of the 46 context-overhead cases, while Supplementary-Material Bloat (SMB) appears in only 3 cases.

Implication: Skill packages should keep always-loaded instructions short and move examples, templates, long checklists, and background material behind explicit lazy-loading triggers.

B. Excessive Procedure (EP)

This is the largest category, accounting for 114 high-confidence efficiency regressions (62.6%). These regressions occur when the skill changes the execution trajectory by adding exploration, implementation work, debugging, or verification. These overheads often arise when skills turn optional activities into mandatory steps, including repository inspection, heavyweight construction procedures, repeated validation, and broad diagnostic checks, even when the concrete task does not require that much work.

Excessive Exploration (EE) occurs in 17 cases (9.3%), where the skill leads the agent to inspect architecture, search for patterns, audit related files, or compare integration points before making the main task change.

Heavy Implementation Pipeline (HIP) represents 30 cases (16.5%) where the skill induces a heavier construction process, such as multi-stage conversion, subprocess workflows, or runtime simulation.

TABLE IV
CLASSIFICATION OF THE 182 HIGH-CONFIDENCE EFFICIENCY REGRESSIONS.

Category / Subcategory		Definition	Count	Percentage
Context Bloat	Skill-Body Context Bloat	The skill body adds enough text to the agent context that each model call becomes substantially more expensive.	43	23.6%
	Supplementary-Material Bloat	The skill directs the agent to load or inspect auxiliary references, templates, examples, or documentation, increasing context or tool-output cost.	3	1.6%
	<i>Subtotal</i>		46	25.3%
Excessive Procedure	Excessive Exploration	The skill leads the agent to perform excessive repository, architecture, or pattern exploration before implementation.	17	9.3%
	Heavy Implementation Pipeline	The skill leads the agent to use a heavier construction pipeline, subprocess workflow, or runtime simulation.	30	16.5%
	Excessive Verification	The skill leads the agent to run excessive or repeated testing, debugging, rebuilding, or checklist verification after implementation.	67	36.8%
	<i>Subtotal</i>		114	62.6%
Dependency Resolution		The skill leads the agent to use a fragile or incompatible runtime dependency that requires installation, configuration, repair, or debugging before the task passes.	22	12.1%
Total			182	100.0%

Excessive Verification (EV) is the largest subcategory, with 67 cases (36.8%), and refers to excessive or repeated tests, rebuilds, debugging attempts, or checklist verification after the main artifact has been produced.

Finding 4: High-confidence efficiency regressions are dominated by Excessive Procedure rather than prompt length alone, and the largest sources within Excessive Procedure are excessive verification and heavy implementation pipelines. At $T = 2.0$, Excessive Procedure accounts for 114 of 182 efficiency regressions (62.6%); within this category, Excessive Verification (EV) contributes 67 cases and Heavy Implementation Pipeline (HIP) contributes 30 cases.

Implication: Skill platforms should model the extra actions that skills induce, and skill authors should condition verification scope and pipeline depth on task uncertainty, change size, and budget rather than prescribing exhaustive workflows by default.

We distinguish Context Bloat from Excessive Procedure by asking where the extra cost comes from. A case is Context Bloat when each model call becomes more expensive because the skill body or supplementary material increases context size, while the overall sequence of task-solving steps remains similar to the reference run. A case is Excessive Procedure when the skill changes the trajectory by adding extra exploration, implementation work, retries, or verification steps. If both effects appear, we assign the subcategory according to the larger cost driver observed in the trajectory.

C. Dependency Resolution (DO)

This category accounts for 22 high-confidence efficiency regressions (12.1%) where the task eventually passes, but the skill leads the agent to use a fragile or incompatible runtime dependency that requires installation, configuration, repair, or debugging. This category covers cases where dependency setup or repair becomes part of the successful trajectory. If the same dependency problem remains unresolved and causes the verifier to fail, it appears in the functional-failure taxonomy as Broken Dependency or Runtime (BDR).

VI. AUTOMATED ATTRIBUTION OF SKILL-INDUCED AGENT FAILURES

As reusable skills become part of agent platforms and skill marketplaces, failure analysis must eventually move beyond one-off manual audits. To explore this use case, we build SKILLTRIAGE, a lightweight LLM-based tool for post-confirmation triage: given a target/reference pair already identified as a skill-induced agent failure, it predicts the high-level category and subcategory, and returns contrastive evidence for that attribution. Figure 4 gives an overview of its input, differential-evidence, and attribution stages. We evaluate SKILLTRIAGE against manually assigned labels for both functional failures and efficiency regressions; the tool is not used to create the taxonomy or replace manual validation.

A. Attribution Tool

The key insight behind SKILLTRIAGE is that the manual taxonomy can be operationalized as an attribution procedure, not merely used as a set of labels. The diagnostic principle is to ask which label best explains both the target outcome, either failure or extra cost, and the divergence between the target-run and reference-run trajectories. Thus each category acts as an evidence checklist: functional failures require skill-scope, runtime, artifact-construction, or task-path evidence, while efficiency regressions require context, procedure, or dependency-cost evidence.

Figure 4 organizes this taxonomy-guided procedure into three stages: input construction, differential-evidence extraction, and attribution.

Input construction. SKILLTRIAGE first normalizes each paired case into a shared task view and two run views: the target-run view records the loaded skill, result, and trajectory, while the reference-run view records the no-skill or matched-skill setup, result, and trajectory. At this stage, deterministic evidence gates check whether the case has the minimum trajectory, skill, and verifier evidence needed for attribution.

Differential evidence. SKILLTRIAGE then turns taxonomy boundaries into explicit differential signals rather than asking the model to infer all evidence directly from raw traces. For functional failures, SKILLTRIAGE computes five differential signals (DS1–DS5) over the functional-evidence surfaces in Figure 4: skill scope, environment, implementation,

and artifact location. DS1 tests whether target-only environment or runtime-state changes plausibly explain the verifier failure. DS2 tests whether such changes are explicitly skill-prescribed or indirectly skill-induced. DS3 tests task-required paths against target write paths. DS4 tests whether the target run produced, stalled before producing, or refused to produce the required artifact. DS5 uses a focused construction-difference check to separate incorrect implementation of a task-required element from required-element omission. For efficiency regressions, SKILLTRIAGE computes phase-level and action-tag cost evidence from step-level token/time/tool records. It splits steps into broad phases such as pre-implementation exploration, implementation or answer-producing pipeline work, and post-implementation verification/debugging, with dependency/setup steps tracked when install, import, or environment-repair signals appear. It also assigns action tags such as exploration (read/search), pipeline (write/data-processing commands), verification (test/debug/rebuild/check), dependency repair (install/import/environment fixes), and reference loading (reading skill-linked templates or documents). The tool then computes aggregate, phase-level, and tag-level cost deltas and uses these signals to support context, procedure, and dependency-cost evidence. The preliminary cost-dominance estimate is used only as an advisory hint; the final label is selected by comparing the high-cost and low-cost step-level evidence and identifying the mechanism that best explains the extra token/time cost.

Attribution. The attribution stage gives the model the taxonomy definitions, normalized input, and extracted differential evidence. It then reasons over candidate labels and selects the root cause that best explains both the target outcome and the target/reference trajectory divergence. For example, when DS3 shows that the target run writes a plausible artifact to a path different from the task-specified path while the reference run writes to the expected path, the attribution stage selects Wrong Artifact Location rather than implementation-content subcategories because the target-run/reference-run divergence is artifact placement, not artifact content. In both branches, the report returns a category, subcategory, natural-language reason, cited skill section or trajectory evidence, and a repair suggestion, so the output supports debugging rather than only label prediction.

B. Evaluation Setup

We evaluate SKILLTRIAGE separately on functional failures and efficiency regressions. For functional failures, we run the tool on all 125 confirmed cases validated by the manual audit; this set includes both with/no-skill and cross-skill failures and covers all seven functional-failure subcategories. For efficiency regressions, we run the tool on all 182 high-confidence PASS/PASS regressions at the primary $T = 2.0$ threshold, including 128 with/no-skill cases and 54 cross-skill cases. For both case sets, we do not rerun the benchmark tasks; instead, we run the attribution procedure with GPT-5.5 [11] three times on the same paired execution evidence for each case and aggregate the three outputs with a 2-of-3 major-

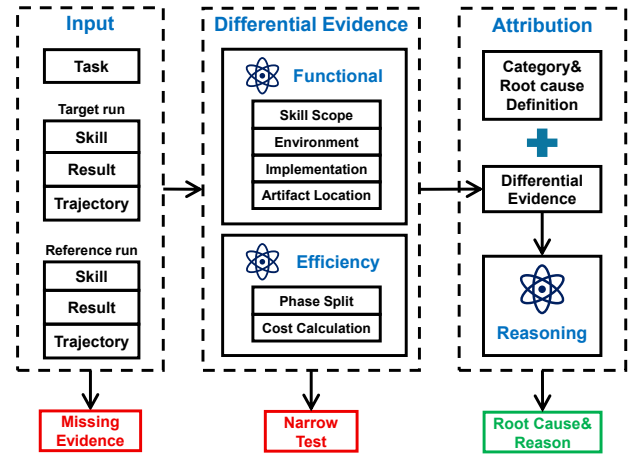


Fig. 4. Workflow of SKILLTRIAGE.

ity vote. Both evaluations use manually assigned attribution labels derived from the corresponding functional-failure and efficiency-regression audits. Majority voting is a standard ensemble strategy for reducing prediction variance [12]; in LLM settings, related self-consistency methods similarly aggregate multiple sampled outputs to improve reliability [13]. We use a 2-of-3 vote because three runs are the smallest odd-number setting that supports a majority decision, reducing single-run attribution variance while keeping evaluation cost modest.

We report three metrics for each branch. Exact subcategory accuracy measures whether the predicted subcategory matches the manual label. Category accuracy measures whether the prediction falls into the correct high-level category. For functional failures, we also report exact subcategory accuracy within Task-Implementation Fault (TIF) cases, the largest functional-failure category. For efficiency regressions, we analogously report exact subcategory accuracy within Excessive Procedure (EP) cases, the largest performance category and a challenging attribution subset, because it requires distinguishing exploration, implementation-pipeline, and verification overhead.

C. Attribution Results

Overall, SKILLTRIAGE provides useful triage-level attribution: it recovers high-level categories for most skill-induced agent failures and identifies exact subcategories in a substantial majority of cases, while the remaining errors concentrate near taxonomy boundaries. Table V reports 2-of-3 majority agreement for functional failure attribution over three independent attribution runs. SKILLTRIAGE matches the human-audit high-level category for 117/125 cases (93.6%) and the exact subcategory for 111/125 cases (88.8%). On TIF cases, where it must distinguish Obstructive Workflow Guidance (OWG), Incorrect Required-Element Fill (IRF), and Required-Element Omission (RRO), it reaches 76/86 (88.4%). The per-root recall inside TIF is 3/4 for OWG, 41/46 for IRF, and 32/36 for RRO.

Table VI reports 2-of-3 majority-vote agreement for efficiency regression attribution against the manual performance labels. Across 182 high-confidence efficiency regressions, SKILLTRIAGE matches the manual performance category for

TABLE V
FUNCTIONAL FAILURE ATTRIBUTION RESULTS.

Subset	Subcategory	Category	TIF subcat.
With/no-skill	35/38 (92.1%)	37/38 (97.4%)	15/17 (88.2%)
Cross-skill	76/87 (87.4%)	80/87 (92.0%)	61/69 (88.4%)
Combined	111/125 (88.8%)	117/125 (93.6%)	76/86 (88.4%)

TABLE VI
EFFICIENCY REGRESSION ATTRIBUTION RESULTS.

Subset	Subcategory	Category	EP subcat.
With/no-skill	98/128 (76.6%)	102/128 (79.7%)	49/69 (71.0%)
Cross-skill	34/54 (63.0%)	43/54 (79.6%)	29/45 (64.4%)
Combined	132/182 (72.5%)	145/182 (79.7%)	78/114 (68.4%)

145/182 cases (79.7%) and the exact subcategory for 132/182 cases (72.5%). Within the EP subset, the tool reaches 78/114 exact subcategory agreement (68.4%), reflecting the difficulty of separating Excessive Exploration (EE), Heavy Implementation Pipeline (HIP), and Excessive Verification (EV).

The residual errors in both branches are boundary errors: the same differential evidence can support neighboring taxonomy labels. For functional failures, the 2-of-3 majority leaves 14/125 exact-subcategory errors, concentrated near the IRF/RRO and Environment Mismatch (EM)/Wrong Artifact Location (WAL) taxonomy boundaries; for example, when a required helper function is implicit in the task, the tool may classify its absence as an incorrect implementation of nearby code rather than as a required-element omission. For efficiency regressions, the 50/182 exact-subcategory errors often arise when one high-cost trajectory contains several plausible cost surfaces: dependency repair inside a verification loop, skill-body cost inflating test steps, or build commands that could be implementation or verification. This error structure suggests that automated attribution should expose the exact skill section, trajectory step, artifact difference, or cost-heavy step used as evidence, not only the final label.

VII. DISCUSSION

A. Future Research Directions

Our findings suggest three directions for making skill reuse safer and more cost-aware.

Skill-task compatibility checks. Because many functional failures arise from topically relevant skills that distort task-required implementation elements or verifier-visible surfaces, future platforms should compare task requirements with candidate skill contents before and during execution. Such checks can extract required fields, APIs, paths, output formats, domain rules, and environment constraints, then warn when a skill introduces conflicting defaults, examples, templates, paths, or package-state assumptions.

Cost-aware skill packaging and selection. Our efficiency regression findings show that skill cost comes from both always-loaded context and induced trajectory steps, not prompt length alone. Future tools should estimate marginal context cost, move task-specific examples or checklists to lazy-loaded references, and predict extra exploration, implementation, verification, or dependency-repair work before selecting a skill.

Budget-aware execution policies. The largest sources within Excessive Procedure are excessive verification and heavy implementation pipelines, suggesting that agents need explicit policies for adapting verification scope and implementation-pipeline depth to task uncertainty, repository size, change risk, and remaining token or execution-time budget. Such policies would preserve careful skill-guided workflows while preventing optional guidance from becoming mandatory work on every task.

B. Threats to Validity

Threats to Internal Validity. Despite the availability of reference materials, such as task specifications, verifier outputs, skill files, generated artifacts, and agent trajectories, analyzing functional failures, efficiency regressions, and their root causes requires manual judgment. The complexity of agent trajectories introduces subjectivity into deciding whether a verifier is narrower than the task, whether a cost change reflects ordinary agent variance, and how the target run diverges from the reference run. To reduce this threat, we excluded ambiguous and verifier-narrow cases and reached group consensus before finalizing the taxonomy labels. Moreover, SKILLTRIAGE provides an independent consistency check by comparing taxonomy-guided triage reports with manually assigned attribution labels for both functional failures and efficiency regressions.

Threats to External Validity. Our study covers agent-skill tasks from multiple application domains, including software engineering, healthcare, manufacturing, cybersecurity, natural science, finance, and mathematics. The dataset includes verifier-graded tasks from SkillsBench and repository-based tasks with deterministic tests from SWE-Skills-Bench, allowing us to analyze skill behavior through reproducible pass/fail and cost signals. This setting provides broader coverage than a single application domain and includes both general task-solving and software-engineering scenarios. However, it is possible that certain findings depend on the task distribution, agent harness, model, or skill ecosystem used in our study and may not directly apply to all settings. To mitigate this threat, we sought to avoid drawing conclusions that are specific to these two benchmarks.

VIII. RELATED WORK

The *Agent Skills* abstraction [8] packages reusable procedural knowledge as on-demand `SKILL.md` documents, extending work on agent capabilities such as skill libraries, executable actions, and tool-oriented tuning [14]–[16]. SkillsBench [1] and SWE-Skills-Bench [2] show that skills can improve average pass rate but also introduce regressions, zero-gain cases and token overhead. We build on these benchmarks but study a different question: which skill contents and induced trajectory changes cause functional failures and efficiency regressions.

Skill loading is also a form of context engineering inside the agent reasoning-and-acting loop [17]. Prior work shows that additional context is not monotonically helpful: models can

ignore middle context, become distracted by irrelevant content, degrade with more documents or longer inputs, and react strongly to prompt format [3]–[6], [18]. These studies motivate skill-loading risk, while our work analyzes how loaded skill context changes executable repository actions, implementation choices, verification behavior, and cost.

Empirical agent studies increasingly analyze failures from traces and benchmarks. MAST [19] builds a taxonomy from agent traces; Agentless [20] and SWE-agent [21] show that scaffold design can materially change outcomes; SWE-bench and its extensions provide evaluation infrastructure for software agents [22]–[26]. Our attribution target is different: we attribute failures and cost regressions to loaded skills under controlled contrastive comparisons, rather than to the base agent or task alone.

Our taxonomy also draws on empirical software-engineering studies of performance bugs, ML bugs, and configuration bugs [27]–[36]. We adapt these perspectives to a new unit of analysis: agent trajectories shaped by skill.

IX. CONCLUSION

This paper studied *skill-induced agent failures*, a failure class induced by skills in LLM agents. By constructing a contrastive analysis dataset, we identified 307 confirmed cases: 125 functional failures and 182 high-confidence efficiency regressions. Our analysis shows that skills induce functional failures through applicability, environment, task-implementation, and artifact mechanisms, and induce efficiency regressions mainly through Excessive Procedure rather than prompt length alone. These findings suggest that skill loading should be treated as a costed and validated configuration decision, supported by compatibility checks, cost prediction, and paired correctness/cost evaluation. Moreover, we developed SKILL-TRIAGE, a taxonomy-guided triage tool that normalizes paired cases, extracts differential evidence, and attributes confirmed failures and regressions to root causes.

REFERENCES

- [1] X. Li, W. Chen, Y. Liu, S. Zheng, X. Chen, Y. He, Y. Li, B. You, H. Shen, J. Sun *et al.*, “Skillsbench: Benchmarking how well agent skills work across diverse tasks,” *arXiv preprint arXiv:2602.12670*, 2026.
- [2] T. Han, Y. Zhang, W. Song, C. Fang, Z. Chen, Y. Sun, and L. Hu, “Swe-skills-bench: Do agent skills actually help in real-world software engineering?” *arXiv preprint arXiv:2603.15401*, 2026.
- [3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.
- [4] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou, “Large language models can be easily distracted by irrelevant context,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 210–31 227.
- [5] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, “Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 25 055–25 083.
- [6] S. Levy, N. Mazor, L. Shalmon, M. Hassid, and G. Stanovsky, “More documents, same length: Isolating the challenge of multiple documents in rag,” *arXiv preprint arXiv:2503.04388*, 2025.
- [7] W. M. McKeeman, “Differential testing for software,” *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.
- [8] B. Zhang, K. Lazuka, and M. Murag, “Equipping agents for the real world with agent skills,” *Anthropic Engineering Blog*, 2025.
- [9] Anomaly, “OpenCode: The open source coding agent,” <https://github.com/anomalyco/opencode>, 2025.
- [10] Anthropic, “Introducing claude opus 4.6,” <https://www.anthropic.com/news/claude-opus-4-6>, 2026.
- [11] OpenAI, “Introducing gpt-5.5,” <https://openai.com/index/introducing-gpt-5-5/>, 2026.
- [12] T. G. Dietterich, “Ensemble methods in machine learning,” in *International workshop on multiple classifier systems*. Springer, 2000, pp. 1–15.
- [13] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” *arXiv preprint arXiv:2203.11171*, 2022.
- [14] G. Wang, Y. Xie, Y. Jiang, A. Mandlkar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *arXiv preprint arXiv:2305.16291*, 2023.
- [15] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better llm agents,” in *Forty-first International Conference on Machine Learning*, 2024.
- [16] R. Yang, L. Song, Y. Li, S. Zhao, Y. Ge, X. Li, and Y. Shan, “Gpt4tools: Teaching large language model to use tools via self-instruction,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 71 995–72 007, 2023.
- [17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [18] M. Levy, A. Jacoby, and Y. Goldberg, “Same task, more tokens: the impact of input length on the reasoning performance of large language models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 15 339–15 353.
- [19] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran *et al.*, “Why do multi-agent llm systems fail?” *Advances in Neural Information Processing Systems*, vol. 38, 2026.
- [20] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying llm-based software engineering agents,” *arXiv preprint arXiv:2407.01489*, 2024.
- [21] J. Yang, C. Jimenez, A. Wetteg, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 50 528–50 652, 2024.
- [22] C. E. Jimenez, J. Yang, A. Wetteg, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157.
- [23] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh *et al.*, “Openhands: An open platform for ai software developers as generalist agents, 2024,” *URL https://arxiv.org/abs/2407.16741*, vol. 2, no. 4, p. 9, 2024.
- [24] D. Zan, Z. Huang, W. Liu, H. Chen, S. Xin, L. Zhang, Q. Liu, L. Aoyan, L. Chen, X. Zhong *et al.*, “Multi-swe-bench: A multilingual benchmark for issue resolving,” *Advances in Neural Information Processing Systems*, vol. 38, 2026.
- [25] S. Miserendino, M. Wang, T. Patwardhan, and J. Heidecke, “Swe-lancer: Can frontier llms earn \$1 million from real-world freelance software engineering?” *arXiv preprint arXiv:2502.12115*, 2025.
- [26] T. Kwa, B. West, J. Becker, A. Deng, K. Garcia, M. Hasin, S. Jawhar, M. Kinniment, N. Rush, S. Von Arx *et al.*, “Measuring ai ability to complete long software tasks,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 92 213–92 266, 2026.
- [27] G. Jin, L. Song, X. Shi, J. Scherpelz, and S. Lu, “Understanding and detecting real-world performance bugs,” *ACM SIGPLAN Notices*, vol. 47, no. 6, pp. 77–88, 2012.
- [28] A. Nistor, T. Jiang, and L. Tan, “Discovering, reporting, and fixing performance bugs,” in *2013 10th working conference on mining software repositories (MSR)*. IEEE, 2013, pp. 237–246.
- [29] M. Selakovic and M. Pradel, “Performance issues and optimizations in javascript: an empirical study,” in *Proceedings of the 38th international conference on software engineering*, 2016, pp. 61–72.
- [30] J. Cao, B. Chen, C. Sun, L. Hu, S. Wu, and X. Peng, “Understanding performance problems in deep learning systems,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 357–369.
- [31] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan, “A comprehensive study on deep learning bug characteristics,” in *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 510–520.
- [32] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, “Taxonomy of real faults in deep learning systems,” in *Proceedings of the ACM/IEEE 42nd international conference on software engineering*, 2020, pp. 1110–1121.
- [33] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, and L. Zhang, “An empirical study on tensorflow program bugs,” in *Proceedings of the 27th ACM SIGSOFT international symposium on software testing and analysis*, 2018, pp. 129–140.
- [34] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, “An empirical study on configuration errors in commercial and open source systems,” in *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, 2011, pp. 159–172.
- [35] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy, and R. Talwadkar, “Hey, you have given me too many knobs!: Understanding and dealing with over-designed configuration in system software,” in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 307–319.
- [36] X. Han and T. Yu, “An empirical study on performance bugs for highly configurable software systems,” in *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2016, pp. 1–10.